Guion del expositor: Carga dinámica

Reparto entre los integrantes (equitativo: 5–5–5–5 diapositivas)

- **Jared Méndez Valencia**: diapositivas 1–5 (portada, objetivos, problema, conceptos y arquitectura Maestro–Trabajador).
- **Daniel del Valle Andrade**: diapositivas 6–10 (protocolo, estático vs. dinámico, algoritmos, diagrama de flujo y cuadro comparativo).
- **Johan Lezama Núñez**: diapositivas 11–15 (paquetes según la velocidad, estrategias, simulación, Gantt y tiempo total).
- **Camila Serrano Trinidad**: diapositivas 16–20 (tamaño del paquete y escalabilidad, clúster de laptops, latencia y CPU/GPU, conclusiones, preguntas y referencias). Johan contesta primero las preguntas de la diapositiva 16.

Reglas del equipo. (1) No se leen las diapositivas: se explican con palabras propias usando las notas como guía. (2) Cada pregunta la contesta primero quien tiene ese tema (tabla de abajo) y los demás pueden complementar. (3) Todos deben poder contestar cualquier pregunta del tema; el código completo está en `codigo_fuente.pdf`.

Mapa de la rúbrica

Criterio	Dónde se cubre
Dominio del tema (25)	Diapositivas 3–12: conceptos, protocolo, algoritmos, diagrama de flujo, cuadro comparativo, estrategias, $M = 2(\lceil N/k \rceil + W)$, T_{ideal}.
Calidad LaTeX/Beamer (20)	`main.tex`: `frame`, `block`, `alertblock`, `exampleblock`, fórmulas `$...$`, tablas con `booktabs`, TikZ.
Entregable gráfico o código (20)	Arquitectura (5), pseudocódigo con `listings` (8), diagrama de flujo (9), gráficas de resultados y escalabilidad (11, 14–16), `codigo_fuente.pdf`.
Aplicación al clúster (20)	Diapositivas 17–18: IP, `hosts.txt`, `mpiexec`, latencia, balanceo CPU/GPU; `cluster_diagnostico.py`.
Preguntas y respuestas (15)	Banco de preguntas de abajo, con responsable por tema.

Diapositiva 1: Portada / cierre

- Presentación del equipo: cada integrante dice su nombre y qué parte expone.
- Frase gancho: "¿Qué pasa cuando un nodo lento frena a uno rápido?" Hoy lo vemos con teoría y con una simulación en una laptop.

Diapositiva 2: Objetivos

- Los cuatro objetivos específicos coinciden con lo que pide el tema: cómo se reparte, algoritmo estático vs dinámico, paquetes según velocidad y estrategias para evitar nodos en espera.

Diapositiva 3: Un nodo lento frena a uno rápido

- Aquí está la idea central de todo el tema: con un reparto igual, el trabajo termina cuando termina el nodo más lento.
- Los nodos rápidos quedan ociosos: se paga hardware que no se aprovecha.

Diapositiva 4: Conceptos y métricas

- Define con tus palabras cada concepto; son el vocabulario del resto de la exposición.
- El ideal es el balance perfecto sin costos de comunicación: el trabajo total entre la suma de velocidades. Si la eficiencia es 100 % nadie espera.

- Vocabulario de sistemas distribuidos que se usará: heterogeneidad (nodos con velocidades distintas), granularidad (tamaño del paquete), overhead de comunicación (mensajes que no son cómputo), cuello de botella (el maestro o el nodo más lento) y escalabilidad (cómo mejora el tiempo al agregar nodos).
- Referencia de métricas: Pacheco & Malensek (2022).

Diapositiva 5: Arquitectura Maestro–Trabajador

- El maestro (rank 0) tiene la cola de tareas y no calcula. Los trabajadores son los nodos, cada uno con su velocidad.
- Dos mensajes: el trabajador dice "estoy libre" y el maestro responde con un paquete.

Diapositiva 6: Protocolo de mensajes

- Tres tipos de mensaje: PEDIR, TRABAJO y FIN. En mpi4py el maestro usa recv con ANY_SOURCE para atender al primero que pida.
- El trabajador es quien manda el ritmo: pide cuando termina. Eso es lo que hace al esquema dinámico.

Diapositiva 7: Estático vs. dinámico

- Izquierda, estático: el maestro decide todo al inicio. Con el mismo número de tareas para todos, el nodo rápido termina antes y espera (zona rayada).
- Derecha, dinámico: con paquetes iguales el nodo rápido pide más veces y hace más tareas; todos terminan casi juntos. El costo es la comunicación.

Diapositiva 8: Algoritmos: estático y dinámico

- Estático: el maestro arma los lotes, los envía con un Scatter y cada trabajador procesa lo suyo; la barrera final es la espera.
- Dinámico: el maestro atiende el primer PEDIR que llegue y responde con un paquete; termina cuando la cola se vacía y envía FIN. El código real está en el archivo aparte balanceo.py.

Diapositiva 9: Diagrama de flujo del maestro dinámico

- Este es el diagrama de flujo del maestro. Se lee de arriba hacia abajo: se inicializa la cola con las N tareas y el contador de trabajadores activos con W. Mientras haya trabajadores activos, el maestro se bloquea en una recepción con ANY_SOURCE y atiende al primero que pida.
- Si todavía hay tareas, responde con un paquete de k tareas; si la cola ya está vacía, responde FIN y descuenta un trabajador activo. Cuando activos llega a cero, el maestro termina.
- Invariante de corrección: cada tarea sale de la cola una sola vez, así que no se repite ni se pierde trabajo. Además, cada trabajador recibe exactamente un FIN.
- Conteo de mensajes: hay $\lceil N/k \rceil$ paquetes más un PEDIR final por trabajador; cada PEDIR tiene su respuesta, por eso $M = 2(\lceil N/k \rceil + W)$. Con $N = 240$, $k = 8$ y $W = 4$ son 68 mensajes; con $k = 4$ serían 128. Esto es el overhead de comunicación que crece cuando k disminuye.

Diapositiva 10: Cuadro comparativo

- Este es el cuadro resumen de la comparación. Subraya: estático gana en simplicidad y pocos mensajes; dinámico gana cuando hay heterogeneidad o cargas impredecibles.

Diapositiva 11: Paquetes según la velocidad

- Respuesta a la pregunta del tema: el maestro nunca conoce las velocidades. El nodo rápido termina antes, pide antes y recibe más paquetes. Las líneas punteadas son el reparto ideal proporcional a la velocidad.
- Guiado: los primeros paquetes son grandes (menos mensajes) y se encogen al final para que nadie se quede con un paquete enorme cuando casi no hay trabajo.

Diapositiva 12: Estrategias para evitar nodos en espera

- Las tres primeras las implementé y las medí. Las otras tres son estrategias de la literatura que quedan como trabajo futuro.
- Prefetch y robo de trabajo se mencionan de nuevo en las conclusiones.

Diapositiva 13: Simulación y diseño experimental

- En una laptop todos los procesos son igual de rápidos, entonces simulo la heterogeneidad: cada trabajador tiene un factor de velocidad y tarda costo dividido entre velocidad. Con mpiexec -n 5 hay 1 maestro y 4 trabajadores.
- Usar sleep hace que los resultados no dependan de cuántos núcleos tenga la laptop. También existe un modo real con carga de CPU.
- Cinco escenarios, cada uno con una pregunta: ¿qué pasa con nodos iguales?, ¿con nodos distintos?, ¿cuál es el mejor tamaño de paquete?, ¿cómo escala?, ¿qué pasa si un nodo se degrada? Mismas tareas (semilla fija) para todas las estrategias.

Diapositiva 14: Quién trabaja y quién espera

- Esta es la diapositiva más importante. Arriba, estático: el nodo 4 trabaja todo el tiempo y los demás están rayados, es decir, esperando.
- Abajo, con dinámico, todos terminan casi al mismo tiempo, cerca de la línea punteada de tiempo ideal.

Diapositiva 15: Tiempo total en dos escenarios

- Con nodos iguales estático y dinámico casi empatan: no hay nada que balancear. Con nodos distintos, el estático tarda 2.41 s y el dinámico 0.92 s.
- A la derecha el caso más duro: un nodo baja de velocidad a mitad de la ejecución y nadie lo sabía de antemano. Ahí ni siquiera el estático ponderado ayudaría; solo el dinámico reacciona.

Diapositiva 16: Tamaño del paquete y escalabilidad

- Paquete: hay un compromiso. Muy chico genera muchos mensajes; muy grande deja a un nodo lento con demasiado trabajo. Con 60 tareas por paquete el tiempo se dispara a más de 2 segundos, igual de malo que el estático.
- El barrido usa una latencia simulada de 3 ms por petición. Entre paquetes de 2 a 8 el tiempo casi empata; la diferencia es ruido de medición.
- Escalabilidad: speedup contra un nodo rápido trabajando solo. El estático casi no mejora porque lo frena el nodo más lento; con 8 trabajadores el dinámico logra 4.9× y el estático solo 1.9×.

Diapositiva 17: Aplicación al clúster de laptops

- Aplicación al clúster: cuatro laptops conectadas por la misma red local, a través de un switch o un router Wi-Fi. El rank 0 es el maestro y vive en una de las laptops; los otros tres ranks son trabajadores. Cada laptop se identifica por su dirección IP; las que aparecen aquí son de ejemplo.
- El archivo hosts.txt le dice a mpiexec en qué IP arrancar cada proceso. Cada laptop necesita la misma versión de Python, mpi4py y MPI, el proyecto en la misma ruta, y acceso por SSH sin contraseña desde el maestro (en Windows se usa MS-MPI). El diagnóstico se corre con cluster_diagnostico.py.
- La heterogeneidad ahora es real: una laptop con GPU, una intermedia y una vieja. Con el modo real ya no se simulan velocidades; cada nodo tarda lo que su hardware le permita. El algoritmo dinámico no necesita saber cuál es cuál: el que termina antes pide antes.

Diapositiva 18: Latencia, paquete y balanceo CPU/GPU

- Latencia. Cada paquete cuesta dos viajes por la red (PEDIR y TRABAJO), de L cada uno, antes de que el trabajador pueda calcular. Si las tareas son muy cortas, ese overhead domina y el cómputo útil se diluye.
- Fórmula: la fracción de sobrecosto f es dos veces L entre dos L más el cómputo del paquete, k por el costo medio entre la velocidad. Para mantener f por debajo de 10 % se despeja k mayor o igual a 18 por L por v entre c barra. Con L de 3 ms, tareas de 10 ms y v igual a 1, k debe ser al menos 5.4. En la simulación casi no hay latencia (L cercana a cero), por eso paquetes chicos empatan; en una red real la cota sube y el paquete óptimo crece.
- Ancho de banda: los mensajes son listas de unos pocos enteros, es decir bytes; por eso lo que importa es la latencia y no el ancho de banda de la red. En Wi-Fi la latencia es mayor y más variable que en Ethernet, así que el paquete óptimo crece.
- CPU y GPU: la velocidad relativa v de cada nodo es su rendimiento (tareas por segundo) dividido entre el del más rápido; no importa si el trabajo lo hace una CPU o una GPU. Con balanceo dinámico no hace falta conocer v: la laptop con GPU pide paquetes más seguido. El estático ponderado sí necesita v medida y no reacciona si el nodo se degrada.
- El script cluster_diagnostico.py mide por nodo la IP, el RTT al maestro, la velocidad relativa y si hay GPU, y sugiere el valor de –velocidades y el tamaño de paquete k.

Diapositiva 19: Conclusiones y trabajo futuro

- Tres ideas para cerrar: estático simple pero frágil; dinámico se adapta; y el tamaño del paquete es la perilla importante (el guiado no siempre gana porque su primer paquete grande puede caer en el nodo lento).
- Regla práctica: si no conozco las velocidades o no son estables, dinámico. Si las conozco y las tareas son parejas, estático ponderado es más simple y barato.
- Sé honesto con las limitaciones: la simulación es una abstracción; en un cluster real habría red, memoria y contención.

Diapositiva 20: ¿Preguntas? · Referencias

- Todas las referencias son de 2021 en adelante, en formato APA 7.
- Preguntas frecuentes: ¿por qué sleep? (reproducible y sin depender de núcleos); ¿por qué el guiado fue peor? (primer paquete grande en el nodo lento); ¿qué pasa si el maestro falla? (trabajo futuro).

Banco de preguntas frecuentes

Responde primero quien aparece entre paréntesis; los demás complementan.

Conceptos y arquitectura (Jared)

- **¿Qué es la heterogeneidad y por qué importa?** Que los nodos tengan velocidades v_i distintas. Con un reparto igual el tiempo total es $T = \max_i T_i$, así que el nodo más lento marca el ritmo y los demás quedan ociosos.
- **¿Speedup y eficiencia son lo mismo?** No. Speedup $S = T_{1\,\text{nodo}}/T$ compara contra un solo nodo. La eficiencia de balanceo del proyecto es $E = T_{\text{ideal}}/T$, con $T_{\text{ideal}} = \sum c_j / \sum v_i$; vale 100 % si todos terminan a la vez y no hay costo de comunicación.
- **¿Por qué el maestro no calcula?** Para atender las peticiones con baja latencia; si calculara, los trabajadores esperarían su turno para pedir trabajo.
- **¿Qué pasa si el maestro falla?** Es un punto único de falla y un posible cuello de botella. Alternativas: robo de trabajo sin maestro (Fernandes et al., 2024) o maestros jerárquicos; está planteado como trabajo futuro.

Protocolo y algoritmos (Daniel)

- **¿Qué hace `ANY_SOURCE`?** Permite que el maestro reciba de cualquier trabajador, así atiende al primero que pide en lugar de seguir un orden fijo.
- **¿Cuántos mensajes se envían?** $M = 2(\lceil N/k \rceil + W)$: cada paquete cuesta un PEDIR y un TRABAJO, y cada trabajador hace un PEDIR final que se contesta con FIN. Con $N = 240$, $k = 8$, $W = 4$ son 68 (con $k = 4$ serían 128).
- **¿Cómo se evita repetir o perder tareas?** La cola vive solo en el maestro y cada tarea sale una vez; el ciclo termina cuando `activos` $= 0$.
- **¿Cuándo conviene estático?** Si las velocidades se conocen y son estables y las tareas son parejas: menos mensajes y más simple. Si no, dinámico.

Simulación y resultados (Johan)

- **¿Por qué `sleep`?** Hace los resultados reproducibles y no dependientes de cuántos núcleos tiene la laptop. También existe el modo `-modo real` con carga de CPU.
- **¿Por qué el guiado salió peor que el dinámico?** En el escenario con nodos distintos (1.05 s contra 0.92 s del dinámico) su primer paquete es grande y puede caer en el nodo lento, que luego retrasa el final. No siempre pierde: con un nodo que se degrada salió mejor (0.65 s contra 0.75 s), porque los paquetes grandes del inicio ahorran mensajes y los chicos del final reparten bien.
- **¿Cómo se eligió el tamaño del paquete?** Barrido de 1 a 60 tareas con una latencia simulada de 3 ms por petición: paquetes chicos aumentan los mensajes y paquetes grandes desbalancean el final. El mejor en nuestra corrida fue 2, aunque entre paquetes chicos (2 a 8) el tiempo casi empata, así que la diferencia es ruido de medición. En la simulación casi no hay latencia; en una red real el paquete óptimo sube.
- **¿Los resultados son confiables?** Mismas tareas (semilla fija) para todas las estrategias y mediana de varias corridas. Las cifras del equipo pueden variar con la laptop; lo que se debe sostener es la tendencia, no el decimal.

Clúster de laptops (Camila)

- **¿Cómo se llevaría esto a laptops reales?** Misma versión de Python, mpi4py y MPI en todas, el proyecto en la misma ruta, SSH sin contraseña, un `hosts.txt` con las IP y `mpiexec -f hosts.txt -n 4 python balanceo.py ...`. El código es el mismo que en la simulación.

- **¿Qué cambia con Wi-Fi?** Aumenta y varía la latencia L, así que el paquete mínimo $k \geq 18\,L\,v/\bar{c}$ crece. Con Ethernet o un switch se pueden usar paquetes más chicos.
- **¿Cómo se maneja CPU y GPU?** Cada nodo tiene una velocidad relativa v_i (rendimiento entre el del más rápido), sin importar qué hardware hace el trabajo. El dinámico no necesita conocerla: la laptop con GPU pide más seguido.
- **¿Cómo se mide la velocidad de cada nodo?** Con `cluster_diagnostico.py`: todos ejecutan la misma carga fija, se compara el tiempo, y también reporta IP, RTT al maestro y si hay GPU.
- **¿Escala a muchos nodos?** Hasta que el maestro se satura: llegan más peticiones por segundo de las que puede atender. Se mitiga con paquetes más grandes, prefetch o robo de trabajo.
- **¿Qué pasa si una laptop se cae?** La versión actual no tolera fallos: MPI normalmente aborta el trabajo. Habría que añadir tiempos límite y reasignar el paquete perdido (trabajo futuro).